A REPORT

ON

CHEFKART - HOME CHEF BOOKING PLATFORM

By

Name of the Student	Registration No.
Harini. K	AP24110010123
Dimple Priya. P	AP24110010215
Adithya. A	AP24110010129
Kishan Blaram. T	AP24110010079
Jathin. M	AP24110010131

Prepared in the partial fulfilment of the

Project Based Learning of Course CSE 201 – Full Stack Development

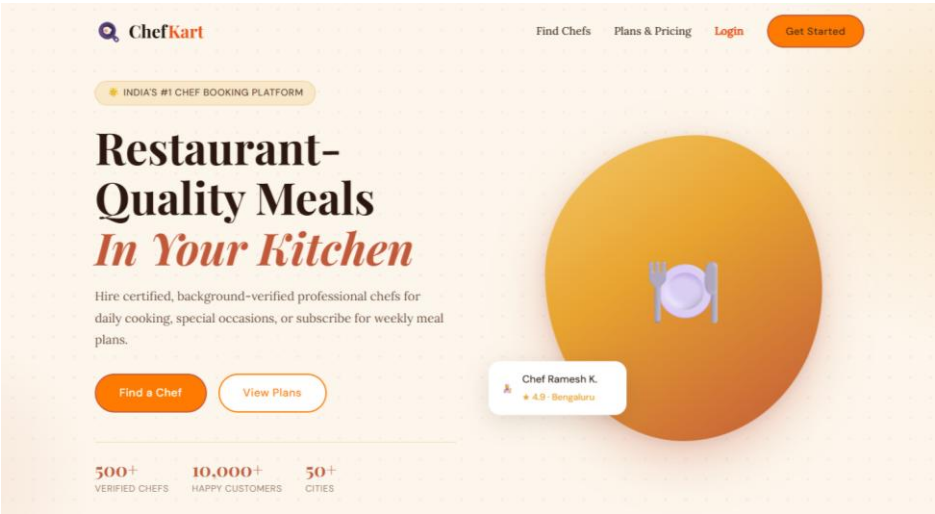
SRM UNIVERSITY, AP

APRIL 2026

INTRODUCTION

ChefKart – Home Chef Booking Platform is a full-stack web application designed to connect users with skilled home chefs in a convenient and efficient manner. In today's fast-paced lifestyle, many individuals and families face challenges in preparing daily meals due to time constraints or lack of cooking expertise. At the same time, many talented home chefs struggle to find reliable opportunities to showcase their skills and earn income.

This platform provides a centralized solution where users can easily search for chefs based on cuisine, availability, and ratings, and book them according to their requirements. It also allows chefs to create professional profiles, manage their availability, and respond to booking requests.



The system is developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**, ensuring scalability, responsiveness, and efficient data handling. By bridging the gap between service seekers and providers, ChefKart aims to simplify the process of hiring home chefs while creating a structured and reliable ecosystem for both users and chefs.

PROJECT DESCRIPTION

ChefKart is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The platform provides a structured system where users can find and book home chefs based on their preferences. It ensures a smooth experience by allowing users to explore chef profiles, schedule bookings, and complete secure transactions. At the same time, chefs can manage their services efficiently, making the platform beneficial for both service seekers and providers. An administrative module oversees system operations, ensuring reliability and proper management of users and bookings.

User Features

- Search for chefs based on cuisine and availability
- View chef profiles with ratings and experience
- Book chefs for specific dates and times
- Make secure payments

Chef Features

- Create and manage profiles
- Update availability
- Accept or reject booking requests
- Earn income through the platform

Admin Features

- Manage users and chefs
- Monitor bookings
- Ensure smooth system operation

PROBLEM STATEMENT

In today's fast-paced lifestyle, many individuals such as working professionals, families, and elderly people face difficulties in preparing daily meals due to time constraints or lack of cooking skills. At the same time, many skilled home chefs struggle to find reliable and consistent job opportunities.
Some of the major issues include:

- Lack of time for daily cooking due to busy schedules
- Difficulty in finding skilled and trustworthy home chefs
- Limited platforms to connect users with available chefs
- Dependence on word-of-mouth, which is unreliable and unstructured
- No centralized system for managing bookings, availability, and payments

Due to these limitations, both users and chefs face inconvenience and missed opportunities. Therefore, there is a need for a centralized digital platform that connects users with home chefs, enabling efficient booking, better service management, and reliable income opportunities.

SCENARIO BASED CASE STUDY

Consider a busy urban environment where many individuals struggle to manage daily cooking due to hectic schedules and lack of time. Meet Riya, a working professional living in a metropolitan city who finds it difficult to prepare meals regularly due to her demanding job. She wants a convenient solution to hire a reliable home chef who can cook healthy meals at her home.

Riya discovers ChefKart – Home Chef Booking Platform, a web-based application designed to connect users with skilled home chefs. Curious about its features, she decides to explore how the platform can simplify her daily routine.

User Registration and Authentication:
Riya creates an account on the platform by entering her details. She logs in securely, ensuring her personal information is protected.

Chef Discovery:
The platform provides a list of available chefs. Riya searches for chefs nearby and filters them based on cuisine preferences such as South Indian and North Indian. She views chef profiles, including experience, pricing, and ratings.

Booking Process:
After selecting a suitable chef, Riya chooses a convenient date and time and sends a booking request through the platform.

Service and Feedback:
The chef accepts the request and provides cooking services at her home. After the service is completed, Riya gives ratings and reviews based on her experience.

Chef Perspective:
Meanwhile, Raj, a skilled home chef, registers on the platform. He creates his profile by adding his specialties, experience, availability, and pricing. He receives booking requests from users, accepts suitable ones, and earns income through the platform.

With ChefKart, both users and chefs benefit from a structured and reliable system. Riya can now manage her meals efficiently, while Raj gains consistent work opportunities. The platform enhances convenience, transparency, and service quality for all users.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the ChefKart – Home Chef Booking Platform, certain system prerequisites must be met.

Software Requirements:

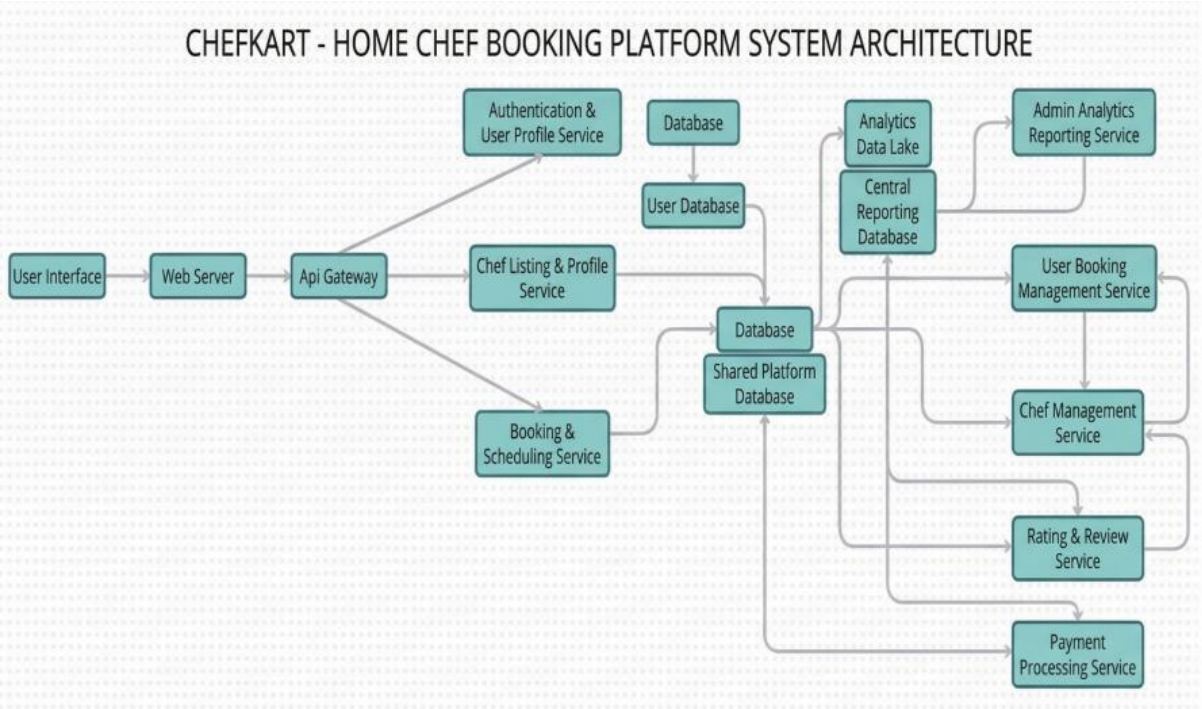
Component	Specification	Purpose
Operating System	Windows 10/11, macOS, or Linux	Cross-platform development and testing
Node.js	v16 or above	Server-side runtime and frontend tooling
npm	v8 or above	Package management for dependencies
React.js	Latest stable	Dynamic and responsive UI development
Express.js	Latest stable	Lightweight REST API framework
MongoDB	v5 or above	NoSQL database for storing application data
Browser	Chrome / Firefox (latest)	UI rendering and testing
VS Code	Latest	Code editor with Git and terminal support
Postman	Latest	API testing during development

Hardware Requirements:

Component	Minimum	Recommended
Processor	Intel Core i3 (8th Gen)	Intel Core i5 or AMD Ryzen 5
RAM	8 GB	16 GB
Storage	1 GB free space	5 GB free space
Display	1366×768	1920×1080 or higher

ARCHITECTURE

The ChefKart – Home Chef Booking Platform follows a full-stack web application architecture built using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system is designed to ensure scalability, efficient data handling, and smooth interaction between users, chefs, and administrators.



Frontend Layer

- Built using React.js
- Provides an interactive and user-friendly interface
- Allows users to search for chefs, view profiles, and book services
- Enables chefs to manage profiles and bookings

Backend Layer

- Built using Node.js and Express.js
- Handles API requests and business logic
- Manages user authentication, chef data, booking operations, and reviews
- Ensures proper communication between frontend and database

Database Layer

- Uses MongoDB
- Stores data related to users, chefs, bookings, payments, and reviews
- Maintains structured and scalable data storage

Authentication & User Management

- Uses JWT (JSON Web Token) for authentication

- Ensures secure login and role-based access (user, chef, admin)
- Manages user profiles and session handling

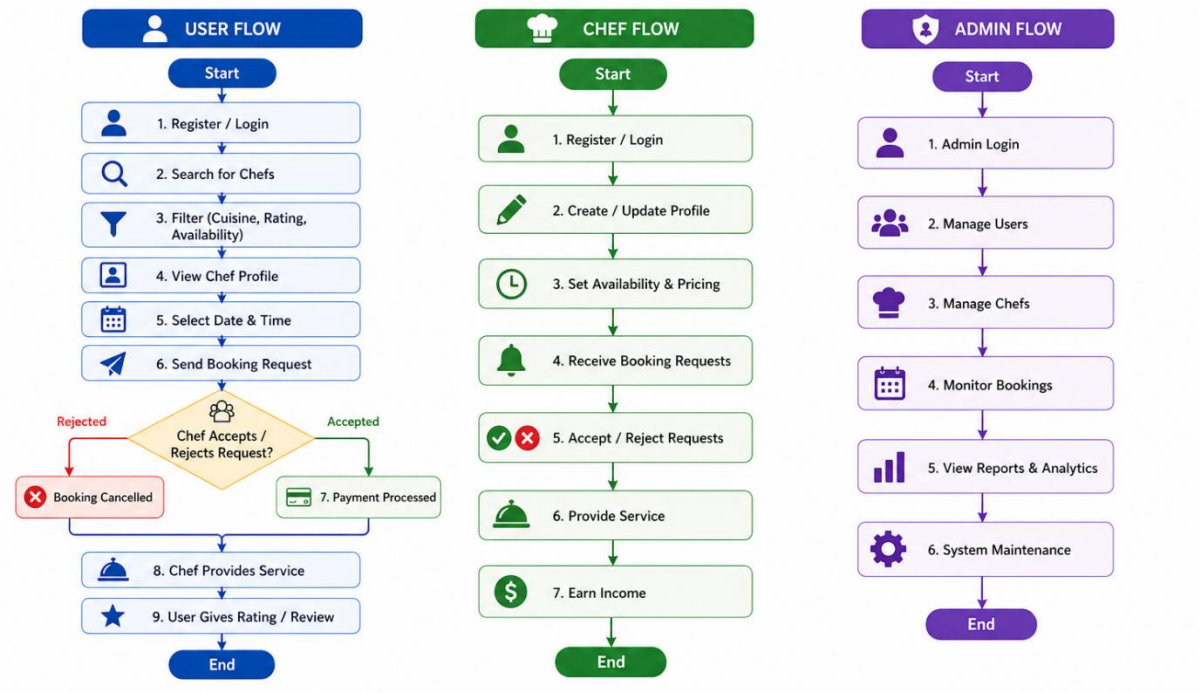
Core Services

- Chef Listing & Profile Service: Handles chef details such as cuisines, experience, and availability
- Booking & Scheduling Service: Manages booking requests, time slots, and scheduling
- User Booking Management: Allows users to create and track bookings
- Chef Management Service: Enables chefs to manage availability and booking responses
- Rating & Review Service: Stores feedback and ratings for quality assurance
- Payment Processing Service: Handles secure transactions and payment records

Admin & Analytics Layer

- Admin monitors users, chefs, and bookings
- Maintains system performance and data integrity
- Generates reports and insights for platform improvement

PROJECT FLOW (SYSTEM FLOW)



Step 1: User Registration

User registers on the platform by entering personal details.

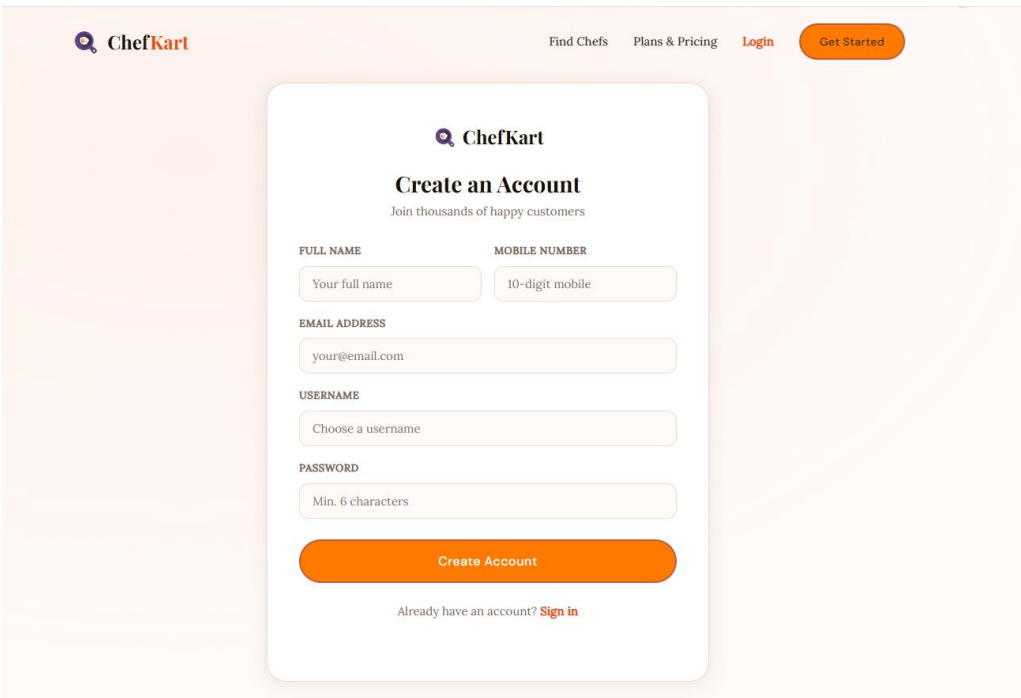


Figure 1: User Registration Page

Step 2: User Login

User logs into the platform securely.

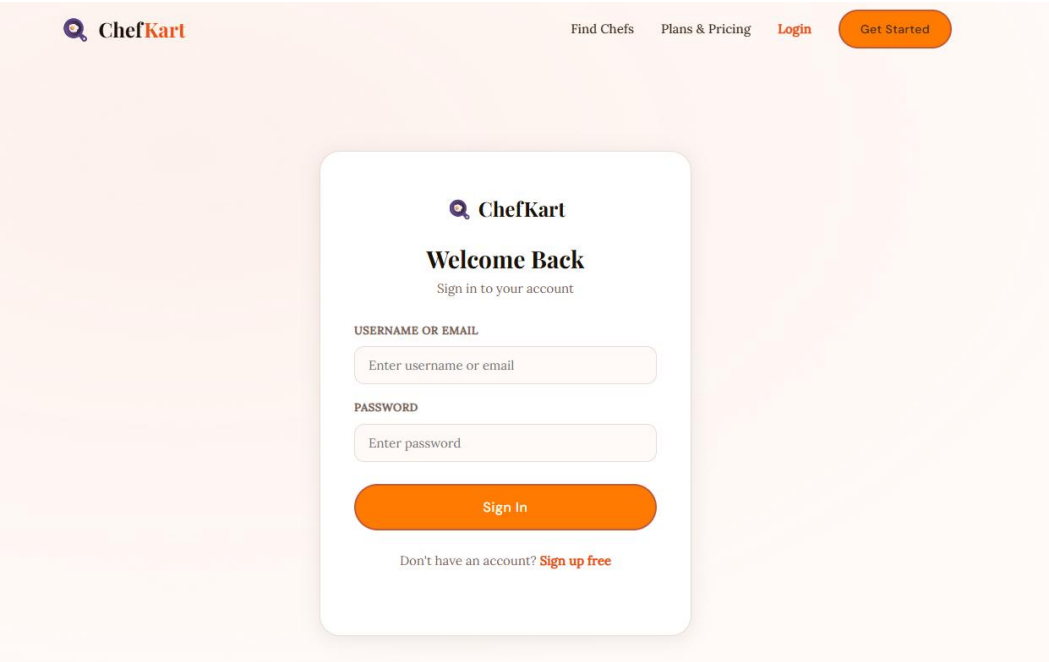


Figure 2: User Login Page

Step 3: Chef Search

User searches for chefs based on preferences.

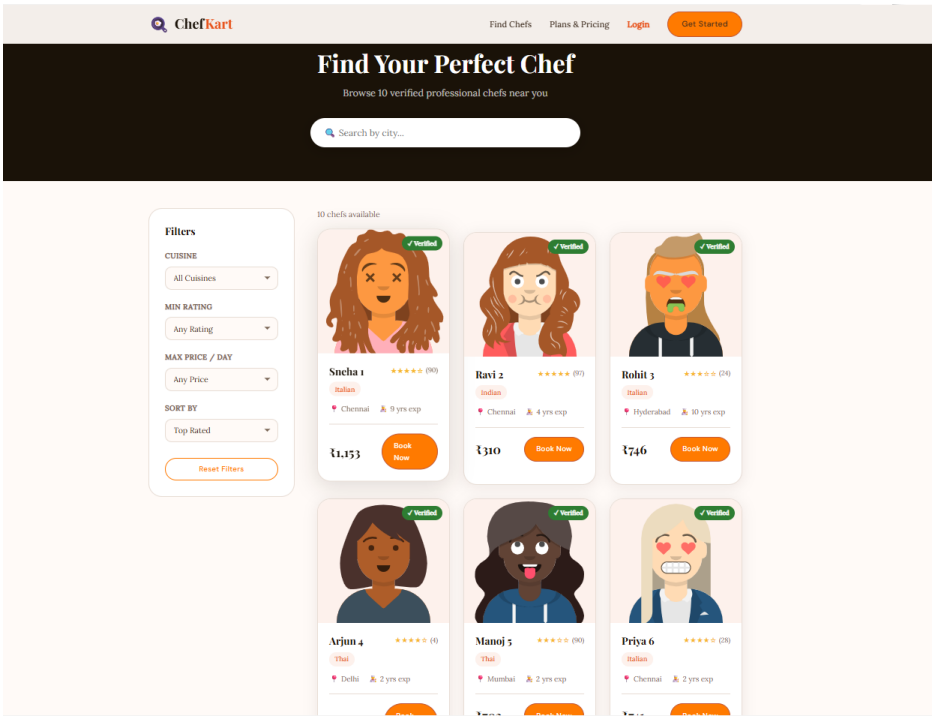


Figure 3: Chef Listing Page

Step 4: View Chef Profile

User views details like experience, cuisine, and ratings.

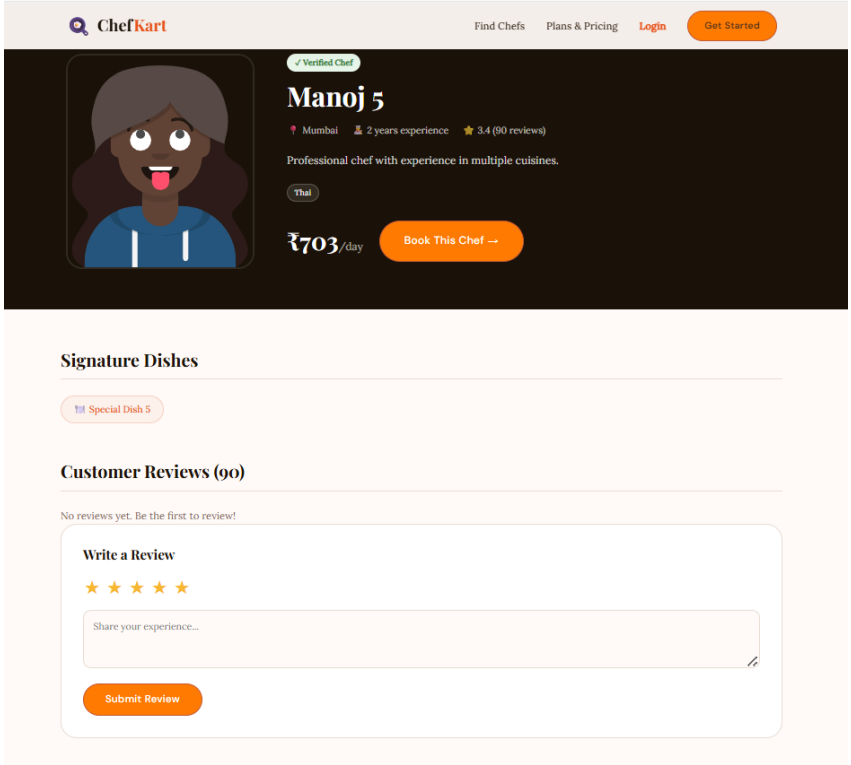


Figure 4: Chef Profile Page

Step 5: Booking

User selects date and time and sends booking request.

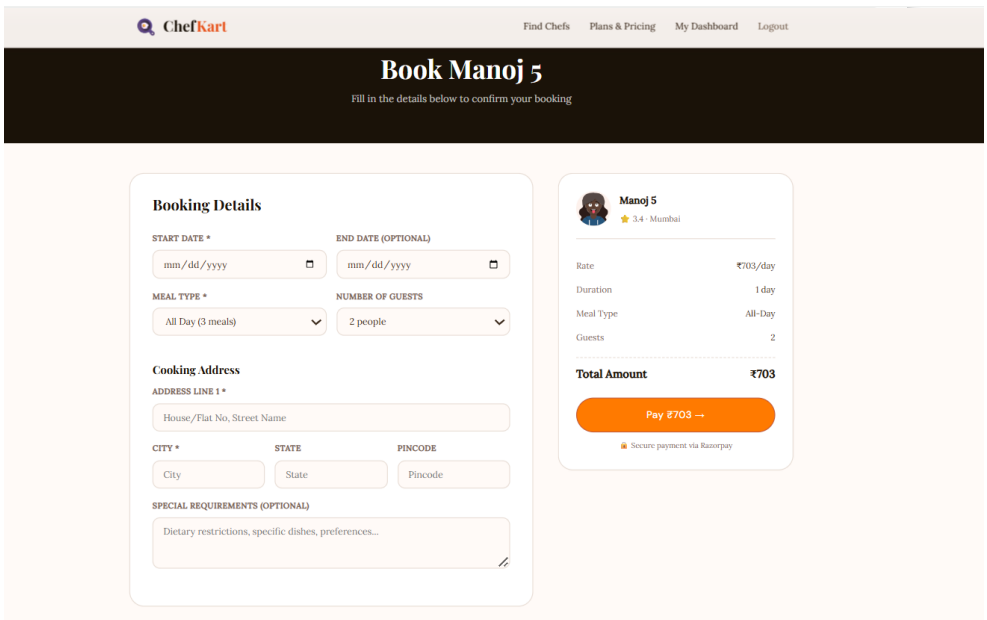


Figure 5: Booking Interface

Step 6: Booking Response
The chef receives the booking request and decides whether to accept or reject it based on availability and schedule.

Step 7: Payment
User completes payment securely.

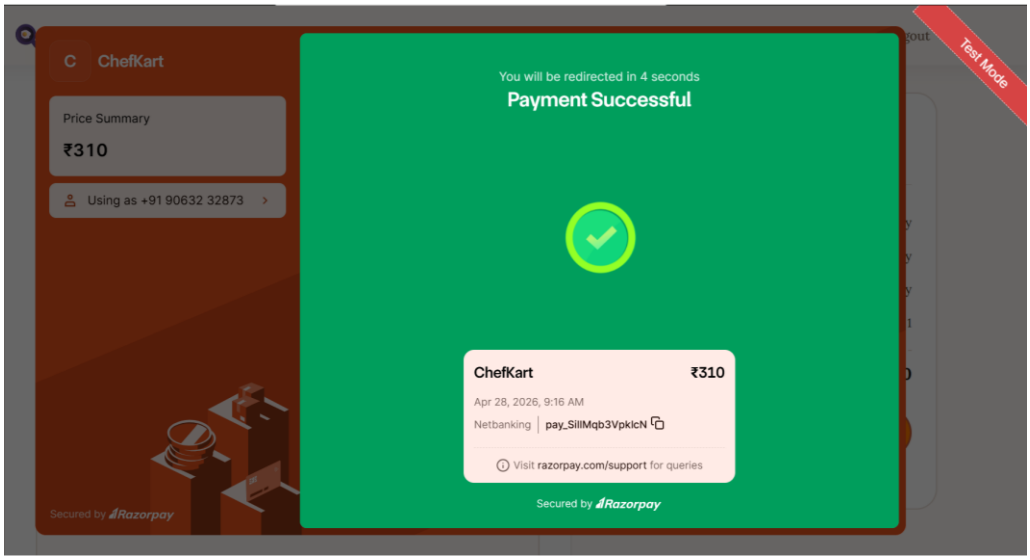


Figure 6: Payment Page

Step 8: Service Completion
After successful booking and payment, the chef arrives at the user's location and completes the cooking service. The system updates the booking status accordingly.

Step 9: Review & Rating
User gives feedback.

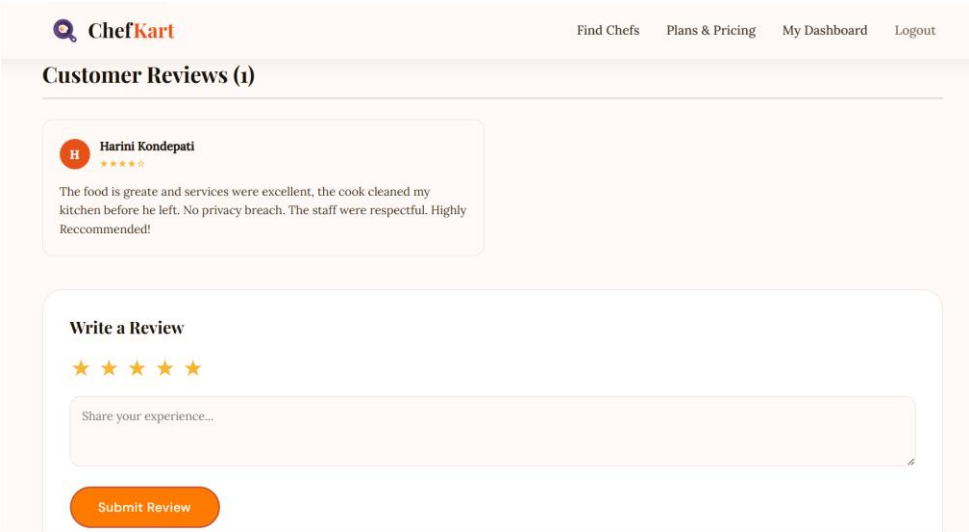


Figure 8: Rating and Review Page

Step 10: Admin Monitoring

Admin manages users and bookings.

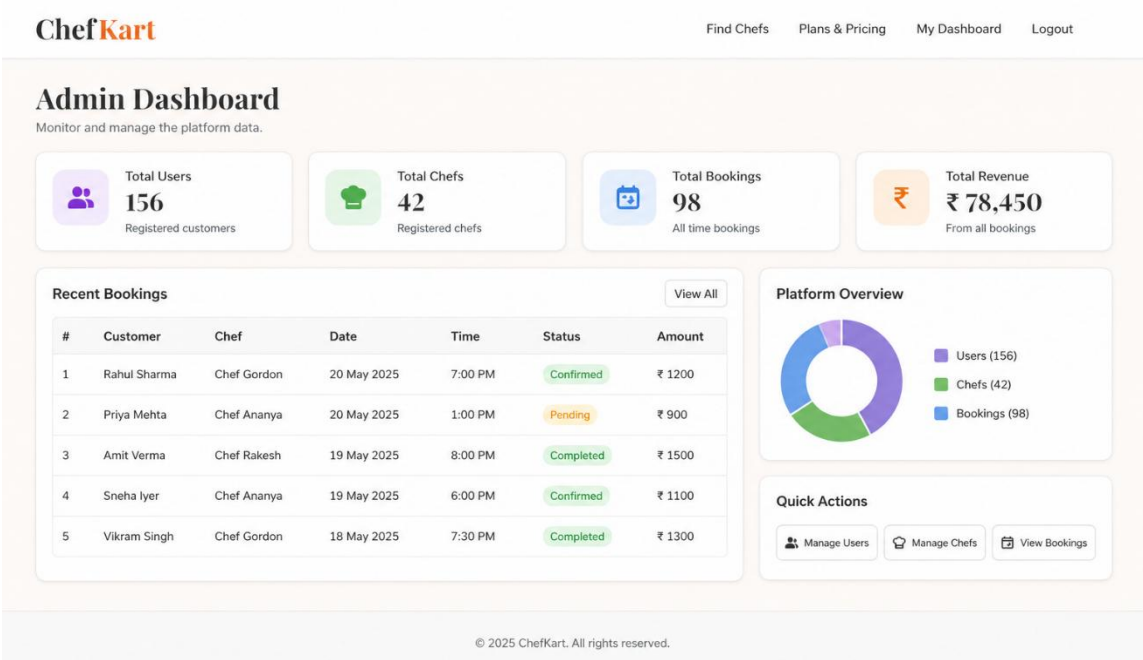
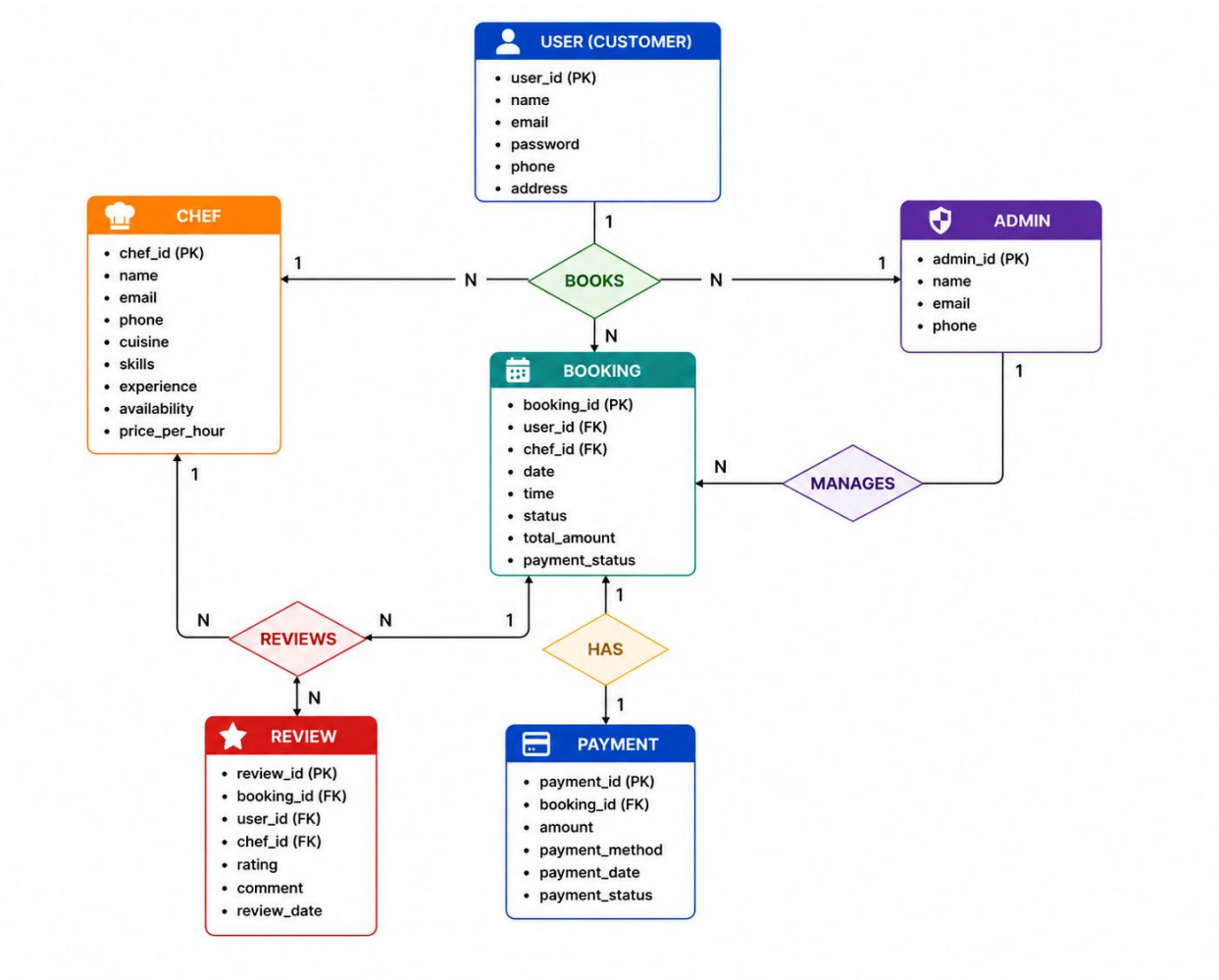


Figure 9: Admin Dashboard

ER DIAGRAM (ENTITY-RELATIONSHIP)

The ChefKart – Home Chef Booking Platform uses an Entity-Relationship (ER) model to represent how users, chefs, bookings, payments, and reviews are structured and related.



1. User Entity

This entity stores information about customers who use the platform to book chefs.

Attributes:

- user_id
- name
- email
- password
- phone
- address

A user can search for chefs, make bookings, and provide reviews after the service.

2. Chef Entity
This entity stores information about chefs who provide cooking services.

Attributes:
- chef_id
- name
- email
- phone
- cuisine
- skills
- experience
- availability
- price_per_hour

A chef can receive multiple booking requests, accept or reject them, and provide services to users.

3. Booking Entity
This entity manages the booking details between users and chefs.

Attributes:
- booking_id
- user_id (FK)
- chef_id (FK)
- date
- time
- status
- total_amount
- payment_status

Each booking represents a service request made by a user to a chef.

4. Payment Entity
This entity stores payment information related to bookings.

Attributes:
- payment_id
- booking_id (FK)
- amount
- payment_method
- payment_date
- payment_status

Each booking has an associated payment that records transaction details.

5. Review Entity
This entity stores feedback given by users after receiving services.

Attributes:
• review_id
• booking_id (FK)
• user_id (FK)
• chef_id (FK)
• rating
• comment
• review_date

A review helps maintain service quality and allows users to share their experience.

6. Admin Entity
This entity stores details of administrators who manage the platform.

Attributes:
• admin_id
• name
• email
• phone

Admin monitors users, chefs, and bookings to ensure smooth system operation.

Relationships
• One User can make multiple Bookings
• One Chef can receive multiple Bookings
• Each Booking is associated with one User and one Chef
• One Booking has one Payment
• One User can give multiple Reviews
• One Chef can receive multiple Reviews
• Each Review is linked to a specific Booking
• One Admin manages multiple Bookings

PRE-REQUISITES

To build this project, knowledge of the following technologies is required.

Frontend
- HTML, CSS
- JavaScript (ES6)
- React.js fundamentals
- React Components and Hooks
- React Router
- Axios for API calls
- TailwindCSS / CSS

Backend
- Node.js fundamentals
- Express.js and routing
- REST API development
- Middleware concepts
- JWT Authentication

Database
- MongoDB document structure
- CRUD operations
- Mongoose ODM
- Schema design

Additional Concepts
- Role-based authentication and authorization
- Client-server architecture
- API integration
- Form validation

REQUIRED TECHNOLOGIES

The ChefKart – Home Chef Booking Platform is developed using modern web technologies that ensure scalability, performance, and security.

Frontend
- React.js
- HTML, CSS
- TailwindCSS / Material UI
- Axios

Backend
- Node.js
- Express.js

Database
- MongoDB
- Mongoose

Authentication & Security
- JWT (JSON Web Token)
- bcrypt

External Services
- Razorpay / Stripe (for payment processing)
- SendGrid (for email services)

Development Tools
- Git
- GitHub
- Visual Studio Code
- Postman / Thunder Client

MVC PATTERN

The ChefKart – Home Chef Booking Platform follows the Model-View-Controller (MVC) architectural pattern, which separates the application into three interconnected layers. This separation improves modularity, maintainability, and scalability of the system.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related operations. It defines the structure of the data and manages interactions with the database. In this project, the models are implemented using Mongoose, which provides schema-based modeling for MongoDB.

The main models include Users, Chefs, Bookings, Payments, and Reviews. These models handle data storage, retrieval, and updates within the application.

Controller Layer

The Controller layer acts as an intermediary between the View and the Model. It processes incoming requests, performs necessary validations or transformations, and interacts with the appropriate models to fetch or update data.

In this project, controllers handle functionalities such as user authentication, chef management, booking operations, payment processing, and review handling. The controller then sends the appropriate response back to the client.

View Layer (Frontend Layer)

The View layer represents the user interface of the application. In ChefKart, it is implemented using React.js, which provides a dynamic and interactive frontend.

Users can interact with the system through various interfaces such as login/signup pages, chef listing pages, booking interfaces, and dashboards. The frontend communicates with the backend through API calls to display and update data.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability
- Scalability: New features can be added easily by extending models, controllers, and views
- Reusability: Code in models and controllers can be reused across different parts of the application
- Easy Testing: Each layer can be tested independently for better reliability
- Organized Structure: Improves project organization and makes development more efficient

SUGGESTED DATABASE COLLECTIONS

Users:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Full name of the user
email	String	Email address (unique)
password	String	Bcrypt hashed password
phone	String	Contact number
address	String	User address

Chefs:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Full name of the chef
email	String	Email address
phone	String	Contact number
cuisine	String	Type of cuisine (e.g., South Indian)
skills	Array	List of cooking skills
experience	String	Years of experience
availability	String	Availability status
price_per_hour	Number	Charges per hour

Bookings:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
user_id	ObjectId	Reference to User
chef_id	ObjectId	Reference to Chef
date	Date	Booking date
time	String	Booking time
status	String	Pending / Accepted / Rejected
total_amount	Number	Total booking cost
payment_status	String	Paid / Unpaid

Payments:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
booking_id	ObjectId	Reference to Booking
amount	Number	Payment amount
payment_method	String	Mode of payment
payment_date	Date	Payment timestamp
payment_status	String	Success / Failed

Reviews:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
booking_id	ObjectId	Reference to Booking
user_id	ObjectId	Reference to User
chef_id	ObjectId	Reference to Chef
rating	Number	Rating given by user
comment	String	User feedback
review_date	Date	Review timestamp

Admin:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Admin name
email	String	Admin email
phone	String	Contact number

PROJECT SETUP AND CONFIGURATION

Creating Project Folder

1. Create a new folder with your project name
2. Inside that folder, create two folders:
3. Name one as **frontend**
4. Name another as **backend**
5. Open the project folder in VS Code

Frontend Setup (React App using Vite)

Open the frontend folder in terminal:

```
npm create vite@latest . -- --template react
```

6. Select **React framework**
7. Select **JavaScript variant**

Navigate to frontend folder:

```
cd frontend
```

Install dependencies:

```
npm install
```

Run the frontend:

```
npm run dev
```

Backend Setup

Open backend folder in terminal:

```
npm init -y
```

Install required packages:

```
npm install express mongoose cors dotenv jsonwebtoken bcrypt
```

Create files:

- server.js

Create folders:

- models
- controllers
- routes
- middleware

BACKEND DEVELOPMENT

Backend Structure:

Controllers

• **authController.js** – Handles user authentication including registration, login, and JWT token generation
• **chefController.js** – Manages chef-related operations such as profile creation, updating details, and availability
• **bookingController.js** – Handles booking operations including creating bookings, updating status, and retrieving booking details
• **paymentController.js** – Manages payment processing and transaction handling

Middleware

• **authenticate.js** – Middleware for authentication and authorization using JWT to protect routes

Models

• **userModel.js** – Schema/model for user accounts
• **chefModel.js** – Schema/model for chef details
• **bookingModel.js** – Schema/model for booking records
• **subscriptionModel.js** – Schema/model for subscription plans (if used in the platform)

Routes

• **authRoutes.js** – Defines API routes for authentication (register, login)
• **chefRoutes.js** – Defines API routes for chef operations (profile, availability, listings)
• **bookingRoutes.js** – Defines API routes for booking operations
• **paymentRoutes.js** – Defines API routes for handling payments
• **subscriptionRoutes.js** – Defines API routes for subscription features

Configuration & Setup

• **configs/** – Contains database configuration and environment setup
• **.env** – Stores environment variables such as database URI and JWT secret
• **server.js** – Entry point of the backend application

DATABASE DEVELOPMENT

1. Configure MongoDB
Install Mongoose. In your Node.js project directory, run:
```
npm install mongoose
```

- Create database connection
- Create schemas and models

Create Database Connection
Before performing any backend operations, the database must be connected properly. The connection is established using Mongoose, which connects the application to MongoDB using a connection URI stored in environment variables. The connection includes error handling to ensure that the server stops if the database connection fails.

Create Schemas
Schemas are defined based on the ER Diagram to structure and manage application data. Mongoose is used to create schema-based models for MongoDB collections.
The main schemas for this application are:

1. UserModel.js
Schema/model for user accounts.
Stores user details such as name, email, password, phone, and address.

2. ChefModel.js
Schema/model for chef profiles.
Stores chef information including cuisine, skills, experience, availability, and pricing.

3. BookingModel.js
Schema/model for booking records.
Manages booking details such as user ID, chef ID, date, time, status, and total amount.

4. PaymentModel.js
Schema/model for payment transactions.
Stores payment details including amount, payment method, and payment status.

5. SubscriptionModel.js
Schema/model for subscription plans (if applicable).
Handles subscription-related data such as plan type and validity.

FRONTEND DEVELOPMENT

The frontend of the **ChefKart** application is developed using **React.js**, providing a dynamic, responsive, and user-friendly interface. It allows users to easily search for chefs, view profiles, make bookings, and manage their activities on the platform.
The frontend communicates with the backend using REST APIs and ensures smooth interaction between users and the system.

Frontend Features

• **User Authentication**
Users can register and log in securely using forms with validation. Authentication is handled using JWT tokens stored on the client side.

• **Chef Search & Filtering**
Users can browse and search for chefs based on cuisine, availability, and preferences.

• **Chef Profile View**
Each chef has a detailed profile displaying experience, skills, pricing, and ratings.

• **Booking System UI**
Users can select date and time and send booking requests through an interactive form.

• **Payment Integration UI**
Frontend integrates with payment gateways (Razorpay/Stripe) for secure transaction processing.

• **User Dashboard**
Users can view their bookings, manage profile details, and track booking status.

• **Admin Dashboard**
Admin can monitor users, chefs, bookings, and overall system activity.

Frontend Technologies Used

• **React.js** – For building interactive UI components
• **React Router** – For client-side routing and navigation
• **Axios** – For API communication
• **CSS / TailwindCSS** – For styling and responsive design
• **JavaScript (ES6)** – Core programming language

Frontend Structure
The front-end is organized into reusable components and pages:

- **Components**
  - Navbar
  - Footer
  - ChefCard
  - ProtectedRoute

- **Pages**
  - Home
  - Login / Register
  - Chef Listing
  - Chef Detail
  - Booking Page
  - User Dashboard
  - Admin Dashboard

Working of Frontend

1. User accesses the website through the browser
2. React Router handles navigation between pages
3. API requests are sent to the backend using Axios
4. Backend responds with data (chefs, bookings, etc.)
5. Data is rendered dynamically on UI components
6. User interactions (booking, login, payment) are processed in real-time

ROLES AND RESPONSIBILITIES

User (Customer)

• Registration and Account Management: Users are responsible for creating an account, providing accurate personal details, and managing their profile.
• Chef Discovery: Users can search for chefs based on preferences such as cuisine, ratings, and availability.
• Booking Services: Users can book chefs by selecting suitable dates and time slots.
• Payment: Users are responsible for completing secure payments for the services booked.
• Feedback: Users can provide ratings and reviews based on their service experience.

Chef

• Profile Management: Chefs create and maintain profiles including skills, cuisines, experience, availability, and pricing.
• Booking Management: Chefs can accept or reject booking requests based on their availability.
• Service Delivery: Chefs are responsible for providing cooking services as per the booking details.
• Availability Updates: Chefs must regularly update their availability and pricing.
• Earnings: Chefs earn income through completed bookings on the platform.

Admin

• System Management: The admin is responsible for overall system operation, ensuring smooth functioning and performance.
• User and Chef Management: The admin manages user and chef accounts, including monitoring activity and resolving issues.
• Booking Monitoring: The admin oversees all bookings and ensures proper service flow.
• Data Management: The admin maintains data integrity and handles system-level configurations.
• Platform Maintenance: The admin ensures the platform runs efficiently and resolves technical issues.

KEY FEATURES

The **ChefKart** platform provides several important features to ensure a seamless experience for users, chefs, and administrators.

User Features

- **User Registration & Login**

Users can securely create an account and log in using authentication mechanisms.

- **Search and Filter Chefs**

Users can browse chefs based on cuisine, availability, ratings, and preferences.

- **View Chef Profiles**

Detailed profiles include experience, skills, pricing, and customer ratings.

- **Booking System**

Users can select date and time and send booking requests to chefs.

- **Online Payment Integration**

Secure payments are enabled using platforms like Razorpay or Stripe.

- **Ratings and Reviews**

Users can provide feedback and rate chefs after service completion.

Chef Features

- **Profile Management**

Chefs can create and update their profiles including skills, cuisines, and pricing.

- **Booking Management**

Chefs can accept or reject booking requests based on availability.

- **Service Delivery**

Chefs provide cooking services at the user's location.

- **Earnings Management**

Chefs earn income through bookings made on the platform.

Subscription Features

- **Subscription Plans**

Users can subscribe to different plans for regular or discounted chef services.

- **Flexible Pricing**

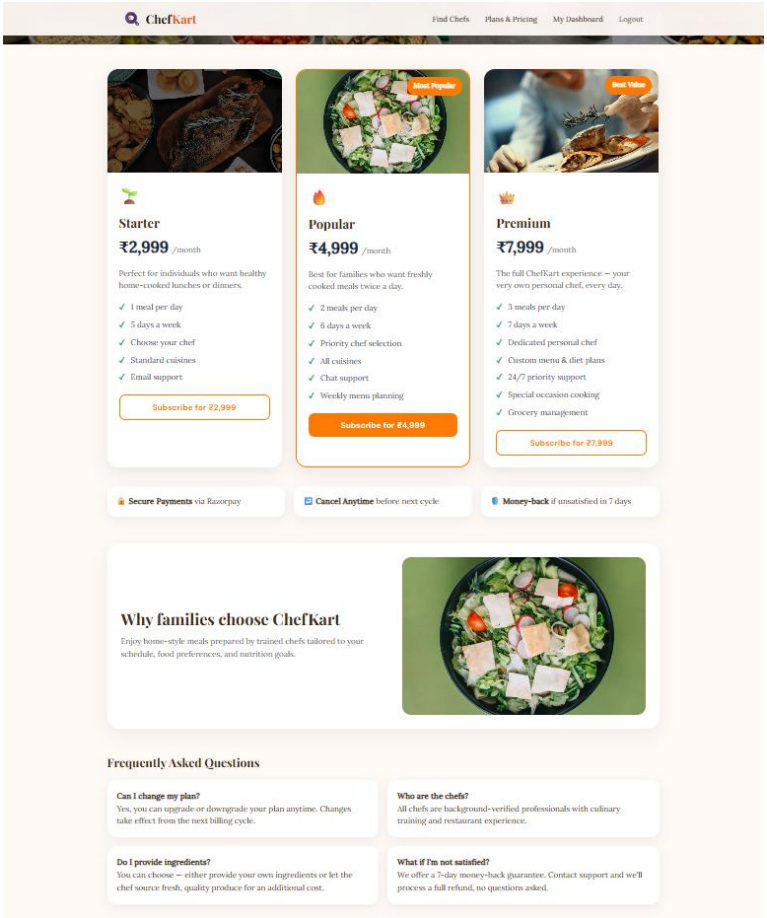
Plans may include weekly, monthly, or customized packages.

- **Priority Booking**

Subscribed users may get priority access to chefs and faster booking confirmation.

- **Cost Benefits**

Subscriptions help users save money compared to one-time bookings.



Admin Features

- **User Management**

Admin can monitor and manage registered users.

- **Chef Management**

Admin oversees chef profiles and verifies service providers.

- **Booking Monitoring**

Admin tracks all booking activities and system usage.

- **System Control**

Admin ensures smooth functioning of the platform.

OPTIONAL ADVANCED FEATURES

- AI-based chef recommendation based on user preferences and past bookings
- Real-time booking updates and notifications
- Live chat system between users and chefs
- Advanced filtering (price range, ratings, cuisine type, experience)
- Integration of online payment gateways (Razorpay / Stripe)
- Push notifications for booking status and updates
- Subscription plans for premium users or chefs
- AI-based rating and review analysis for better recommendations

LEARNING OUTCOMES

By completing this project, the team gained the following learnings:

- Full-stack MERN application development
- Designing a service-based platform (Chef booking system)
- Implementation of authentication and authorization using JWT
- Building RESTful APIs using Node.js and Express.js
- Database schema design using MongoDB and Mongoose
- Handling real-time booking and payment workflows
- Integration of third-party services like payment gateways
- Frontend development using React.js and responsive UI design
- Debugging, testing, and API validation using tools like Postman
- Team collaboration and version control using Git and GitHub

PROJECT REPOSITORY (GITHUB)

GitHub Link: https://github.com/AP24110010123/Group3-ChefKart